

Programmation fonctionnelle 2
Preuves et programmes
CM9 : Types entiers (encore) et propositions
inductives

Pierre Rousselin

Université Sorbonne Paris Nord

mars 2026

Retour sur **positive**

Les autres types d'entiers en binaire

Un exemple de proposition inductive

Définition et notations

```
(* Dans Corelib.Numbers.BinNums *)
```

```
Inductive positive : Set :=  
  | xI : positive -> positive  
  | x0 : positive -> positive  
  | xH : positive.
```

```
(* Dans Corelib.BinNums.PosDef *)
```

```
Notation "1" := xH : positive_scope.
```

```
Notation "p ~ 1" := (xI p) : positive_scope.
```

```
Notation "p ~ 0" := (x0 p) : positive_scope.
```

L'entier 6 peut s'écrire :

- ▶ `x0 (xI xH)` (sans notations, avec constructeurs)
- ▶ ou dans l'autre sens (un peu plus naturel) avec les notations `1~1~0`
- ▶ Le plus important ici est de comprendre que c'est un type à trois constructeurs :
 - ▶ `xH` pour l'entier 1
 - ▶ `x0` pour rajouter un 0 au bout (en bit de poids faible) d'un `positive` (multiplier par 2)
 - ▶ `xI` pour rajouter un 1 au bout (en bit de poids faible) d'un `positive` (multiplier par 2 puis ajouter 1)

Le successeur

```
Fixpoint succ (p : positive) :=
  match p with
  | 1 => 1~0
  | q~0 => q~1
  | q~1 => (succ q)~0
  end.
(* Par définition : *)
Lemma succ_1 : succ 1 = 1~0.
Proof. reflexivity. Qed.
Lemma succ_x0 (p : positive) : succ p~0 = p~1.
Proof. reflexivity. Qed.
Lemma succ_xI (p : positive) : succ p~1 = (succ p)~0.
Proof. reflexivity. Qed.
```

Successeur : propriétés immédiates

- ▶ Contrairement à `S` dans `nat` qui est un constructeur, `succ` dans `positive` n'est pas un constructeur.
- ▶ On n'a pas « gratuitement » `discriminate` et `injection` pour `succ` (mais on les a avec les constructeurs `xH`, `xI` et `xO`).
- ▶ `Lemma neq_succ_1 (p : positive) : succ p <> 1.`

`Proof.`

`unfold "<>".`

`intros H.`

`Fail discriminate H.`

`destruct p as [p' | p' |].`

`- simpl in H. discriminate H. (* xI _ n'est pas égal à xH *)`

`- simpl in H. discriminate H. (* xO _ n'est pas égal à xH *)`

`- simpl in H. discriminate H. (* xO _ n'est pas égal à xH *)`
`(* intros H; destruct p; discriminate H. *)`

`Qed.`

- ▶ Conclusion : on doit détruire le `positive`, pour faire une étape de calcul.

Successesseur, encore

► **Lemma** `neq_succ_diag_l` (`p : positive`) : `succ p <> p`.
Proof.

Successeur, encore

- **Lemma** `neq_succ_diag_l` (`p : positive`) : `succ p <> p`.

Proof.

```
destruct p as [p' | p' |].
```

```
- simpl. intros H. discriminate H.
```

```
- simpl. intros H. discriminate H.
```

```
- simpl. intros H. discriminate H.
```

Qed.

- Preuve courte : sans nommer les arguments (qu'on n'utilise pas explicitement ensuite) dans `destruct`, laisser `discriminate` faire les `intros` tout seul et chercher lui même dans le contexte une égalité discriminable :

```
Lemma neq_succ_1' (p : positive) : succ p <> 1.
```

```
Proof. destruct p; discriminate. Qed.
```

Injectivité de succ

Ça se complique franchement...

Lemma succ_inj (p q : positive) :

succ p = succ q -> p = q.

Proof.

(On raisonne par induction sur p en généralisant l'énoncé sur q :*)*

induction p **as** [p' IH | p' IH |] **in** q |- *;

(et on raisonne par cas dans chaque sous-cas : *)*

destruct q **as** [q' | q' |].

- *(* p = p'~1 et q = q'~1 : *)*

simpl. intros H. *(* H : (succ p')~0 = (succ q')~0 *)*

injection H. *(* H : succ p' = succ q' *)*

(IH : forall q, succ p' = succ q -> p' = q*

*donc (IH q' H) a le type : p' = q' *)*

rewrite (IH q' H). **reflexivity**.

- **simpl. intros** H. *(* H : (succ p')~0 = q'~1 *)*

discriminate H.

- **simpl. intros** H. **injection** H. *(* H : succ p' = 1*)*

(On sait que c'est impossible d'après neq_succ_1 : *)*

exfalso. apply (neq_succ_1 p'). **exact** H.

- *(* autres cas à peu près similaires... *)*

Retour sur `positive`

Les autres types d'entiers en binaire

Un exemple de proposition inductive

N : entiers positifs ou nuls

- ▶ Facile : deux constructeurs
- ▶ `Inductive N : Set := NO | Npos (p : positive).`
- ▶ Entiers naturels comme `nat`.
- ▶ `nat` est plus simple pour prouver seulement 0 et S
- ▶ N est meilleur pour le calcul et la taille en mémoire

Entiers relatifs

- Première tentative naturelle :

```
Inductive Rel :=  
| plus (n : nat)  
| moins (n : nat).
```

- Problème :

```
Lemma problème : plus 0 <> moins 0.
```

```
Proof.
```

```
  discriminate.
```

```
Qed.
```

- En maths on dirait « c'est pas grave, on passe au quotient. »
- En Rocq, c'est plus compliqué... On n'aime pas trop casser l'égalité pour si peu.

Utiliser `positive`

- ▶ Les entiers de Rocq :
`Inductive Z : Set :=`
 - `| Z0`
 - `| Zpos (p : positive)`
 - `| Zneg (p : positive).`
- ▶ Premier avantage de `positive` : pas de 0, donc pas de représentations multiples de 0.
- ▶ Et les mêmes autres avantages que pour `N`.
- ▶ Problème :
 - ▶ `induction` (de base) équivalent à `destruct` (normal, aucun argument de type `Z` dans les constructeurs)
 - ▶ Définitions compliquées, certaines preuves très dures à la main.
- ▶ Notre sauveur dans ce cours où on ne va pas non plus passer notre temps à écrire des preuves sur `Z : lia` (pour « *linear integer arithmetic* »)

Retour sur `positive`

Les autres types d'entiers en binaire

Un exemple de proposition inductive

Souvenirs, souvenirs

Exercice de Programmation Fonctionnelle 1 (Sergueï Lenglet).

Exercice 1 (Dérivations) :

Soit un ensemble E défini par induction de la manière suivante :

$$\frac{}{(0,0) \in E} \text{ ZZ}$$

$$\frac{(x,y) \in E}{(x-1,y+1) \in E} \text{ UN}$$

$$\frac{(x,y) \in E}{(x+2,y-2) \in E} \text{ DEUX}$$

Question 1 – Est-ce que les éléments suivants sont dans E ? Si oui, donner une dérivation.

1. $(-3, 3)$
2. $(1, 1)$
3. $(1, -1)$
4. $(0, 1)$

Souvenirs, souvenirs

Exercice de Programmation Fonctionnelle 1 (Sergueï Lenglet).

Exercice 1 (Dérivations) :

Soit un ensemble E défini par induction de la manière suivante :

$$\frac{}{(0,0) \in E} \text{ ZZ}$$

$$\frac{(x,y) \in E}{(x-1,y+1) \in E} \text{ UN}$$

$$\frac{(x,y) \in E}{(x+2,y-2) \in E} \text{ DEUX}$$

Question 1 – Est-ce que les éléments suivants sont dans E ? Si oui, donner une dérivation.

1. $(-3, 3)$
2. $(1, 1)$
3. $(1, -1)$
4. $(0, 1)$

C'était le bon temps ! On passait pas notre temps à prouver que
 $n + 1 = 1 + n$!

Définition inductive

- ▶ Il n'y a pas d'ensembles a priori en Rocq.
- ▶ Plutôt qu'un ensemble de couples d'entiers relatifs, on considère un prédicat « à deux variables » : $E : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Prop}$.
- ▶ Il est défini inductivement :

Inductive $E : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Prop} :=$

| $\text{ZZ} : E\ 0\ 0$

| $\text{UN } (x\ y : \mathbb{Z})\ (h : E\ x\ y) : E\ (x - 1)\ (y + 1)$

| $\text{DEUX } (x\ y : \mathbb{Z})\ (h : E\ x\ y) : E\ (x + 2)\ (y - 2).$

- ▶ Important : passer du temps pour se convaincre que notre formulation correspond bien au problème posé.

Prouver que $(-3, 3)$ est dans E

- ▶ Avec une preuve vers l'arrière :

Lemma e1q11 : E (-3) 3.

Proof.

apply (UN (-2) 2).

apply (UN (-1) 1).

apply (UN 0 0).

exact ZZ.

Qed.

- ▶ Avec une preuve vers l'avant (remarquer que Rocq n'a alors pas besoin qu'on lui fournisse x et y) :

Lemma e1q11' : E (-3) 3.

Proof.

specialize ZZ **as** H.

(ZZ est dans le contexte local sous le nom H *)*

apply UN **in** H. *(* H : E (-1) 1 *)*

apply UN **in** H. *(* H : E (-2) 2 *)*

apply UN **in** H. *(* H : E (-3) 3 *)*

exact H.

Qed.

Autres questions

- Prouver que $(1, -1)$ est dans E :

Lemma e1q13 : $E \ 1 \ (-1)$.

Proof.

apply (DEUX $(-1) \ 1$).

apply (UN $0 \ 0$).

exact ZZ.

Qed.

- Les autres ? Pas évident sans lemme intermédiaire...

Nouvelle question

Lemma e2q2 (x y : Z) : E x y $\rightarrow$ x + y = 0.

Proof.

intros H.

induction H as [|x y h IH | x y h IH].

- reflexivity.

- (* On suppose h : E x y et IH : x + y = 0,
on doit prouver (x - 1) + (y + 1) = 0 *)

lia.

- (* On suppose h : E x y et IH : x + y = 0,
on doit prouver (x + 2) + (y - 2) = 0 *)

lia.

Qed.

Beaucoup plus dur (on y reviendra)

Lemma `e2q2'` $(x\ y : \mathbb{Z}) : x + y = 0 \rightarrow E\ x\ y.$

Proof.

```
induction x using Z.peano_ind in y |- *.  
- simpl. intros H. rewrite H. exact ZZ.  
- intros H.  
  replace (Z.succ x) with ((x + 2) - 1) by lia.  
  replace y with (((y + 1) - 2) + 1) by lia.  
  apply UN. apply DEUX. apply IHx. lia.  
- intros H.  
  replace (Z.pred x) with (x - 1) by lia.  
  replace y with ((y - 1) + 1) by lia.  
  apply UN. apply IHx. lia.
```

Qed.